

1 - Laboratorio: regresión lineal en práctica

Ejercicio práctico para entrenar y evaluar un modelo de regresión lineal usando Python. Se implementa un pipeline básico que incluye preparación de datos, ajuste del modelo, diagnóstico residual y cálculo de métricas de regresión (MAE y RMSE).

Marco Teórico — Laboratorio de Regresión Lineal

Objetivo del laboratorio

El objetivo de este laboratorio es aplicar un modelo de aprendizaje supervisado de tipo regresión para resolver un problema de negocio realista: estimar el precio de una vivienda a partir de sus características.

A lo largo de la práctica, se busca que el estudiante:

- Comprenda cuándo un problema es de **regresión**
 - Aplique correctamente el flujo típico de un modelo supervisado
 - Interprete métricas de error
 - Analice los coeficientes del modelo para obtener conclusiones de negocio
-

1 Comprensión del problema: ¿por qué regresión?

En este caso, la variable objetivo es **el precio de una vivienda**, que es un **valor numérico continuo**.

Esto define claramente el problema como uno de **regresión**, ya que:

- No se busca clasificar en categorías
- Se desea predecir un valor real (por ejemplo, \$150.000)

☞ Por lo tanto, un modelo de **regresión lineal** es una primera aproximación razonable.

2 Generación del dataset sintético

En la práctica se utiliza un **dataset sintético**, lo cual es habitual en entornos educativos para:

- Controlar las relaciones entre variables

- Evitar problemas de calidad de datos reales
- Comprender mejor el comportamiento del modelo

Variables predictoras (X)

Las variables simuladas representan atributos típicos del mercado inmobiliario:

- `tamaño_m2`: superficie de la vivienda
- `habitaciones`: cantidad de habitaciones
- `antigüedad_años`: años desde la construcción

Estas variables se eligen porque **tienen una relación lógica con el precio**, lo cual facilita la interpretación posterior.

Variable objetivo (y)

- `precio`: se construye como una **combinación lineal** de las variables anteriores más un término de **ruido aleatorio**, para simular la variabilidad real del mercado.

Esto permite modelar un escenario realista donde:

- El tamaño y las habitaciones aumentan el precio
- La antigüedad lo reduce
- Existen factores no observados (ruido)

3 Preparación de variables (X e y)

Antes de entrenar cualquier modelo supervisado, es fundamental separar:

- **X (features)**: variables predictoras
- **y (target)**: variable que queremos predecir

Esta separación es clave porque:

- El modelo aprende una relación matemática entre X e y
- Permite evaluar el desempeño de forma correcta

🔗 En esta práctica, los estudiantes deben identificar explícitamente qué columnas cumplen cada rol.

4 División Train / Test

Para evaluar si el modelo **generaliza correctamente**, se divide el dataset en:

- **Conjunto de entrenamiento (80%)**
- **Conjunto de test (20%)**

Esta división permite:

- Entrenar el modelo solo con una parte de los datos
- Evaluarlo sobre datos no vistos

Esto simula el escenario real donde el modelo debe predecir nuevos casos.

5 Entrenamiento del modelo de Regresión Lineal

La **regresión lineal** asume que existe una relación del tipo:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 \quad y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

Donde:

- Cada coeficiente representa el impacto de una variable
- El intercepto (β_0) es el valor base cuando todas las variables valen cero

Este modelo se elige porque:

- Es simple
 - Es interpretable
 - Funciona como baseline en problemas de regresión
-

6 Predicción y métricas de evaluación

Una vez entrenado el modelo, se generan predicciones sobre el conjunto de test y se evalúa el error usando métricas estándar de regresión:

Métricas utilizadas

- **MAE (Mean Absolute Error)**

Error promedio en unidades reales (pesos)

- **MSE (Mean Squared Error)**

Penaliza más los errores grandes

- **RMSE (Root Mean Squared Error)**

Error típico esperado (misma unidad que el target)

- **R² (Coeficiente de determinación)**

Proporción de la variabilidad del precio explicada por el modelo

Estas métricas permiten evaluar tanto:

- Precisión
- Estabilidad
- Capacidad explicativa

7 Interpretación de coeficientes

Una de las principales ventajas de la regresión lineal es su **interpretabilidad**.

Cada coeficiente indica:

- Cuánto cambia el precio esperado ante un aumento unitario de esa variable
- Manteniendo las demás constantes

Ejemplo conceptual:

- Si el coeficiente de tamaño_m2 es 1200, significa que **cada metro cuadrado adicional aumenta el precio en promedio \$1200**

Ordenar los coeficientes por su valor absoluto permite identificar:

- Las variables con mayor impacto
 - Qué factores son más relevantes desde el punto de vista del negocio
-

8 Gráficos de diagnóstico

Los gráficos ayudan a validar los supuestos del modelo:

Actual vs Predicho

- Permite visualizar qué tan cerca están las predicciones de los valores reales
- Idealmente los puntos deberían alinearse con la diagonal

Análisis de residuales

- Un modelo adecuado debería mostrar residuales:
- Centrados en cero
- Sin patrones evidentes

Si aparecen patrones, puede indicar:

- Relaciones no lineales
- Variables omitidas
- Necesidad de modelos más complejos

2 - Laboratorio: árboles de decisión y Random Forest

Ejercicio práctico para entrenar y evaluar modelos de árboles de decisión y Random Forest, interpretar la importancia de características y comparar su desempeño con modelos lineales.

Unidad: Modelos Basados en Árboles y Ensamble

1. El Problema de Negocio: Predicción de Churn

En el laboratorio, trabajamos sobre un problema de **clasificación binaria**. El *churn* (fuga de clientes) es un fenómeno donde el objetivo no es solo predecir quién se va, sino entender **por qué** se va.

Los modelos basados en árboles son ideales aquí porque:

- Manejan relaciones no lineales.

- Son capaces de capturar interacciones entre variables (ej. "si es joven y tiene pocos productos, la probabilidad de churn sube").
- Ofrecen una interpretación visual directa.

2. El Árbol de Decisión: La Base Atómica

Un Árbol de Decisión particiona el espacio de los datos en rectángulos cada vez más pequeños, buscando la máxima **pureza** en cada nodo.

Conceptos Clave:

- **Criterio de Partición:** El algoritmo decide dónde cortar una variable (ej. `tenure_years < 2.5`) usando métricas como la **Impureza de Gini** o la **Entropía**.
- La fórmula de Gini para un nodo t es:

$$G(t) = 1 - \sum_{i=1}^C p(i|t)^2$$

Donde $p(i|t)$ es la probabilidad de la clase i en ese nodo.

- **Sobreajuste (Overfitting):** Un árbol sin límites crecerá hasta memorizar cada fila del dataset. Por eso usamos `max_depth` (profundidad) y `min_samples_leaf` para obligar al árbol a generalizar.

Shutterstock

3. Random Forest: El Poder de la Multitud

Si un árbol es un "experto" que a veces se equivoca por ser muy específico, el **Random Forest** es un comité de expertos. Es un modelo de **Ensamble** de tipo **Bagging** (*Bootstrap Aggregating*).

¿Cómo funciona?

1. **Bootstrap:** Crea múltiples sub-datasets tomando muestras aleatorias con reemplazo del original.
 2. **Muestreo de Variables:** En cada partición de cada árbol, solo se le permite ver un subconjunto aleatorio de variables (columnas). Esto "descorrelaciona" los árboles.
 3. **Votación:** La predicción final es el promedio (regresión) o la mayoría (clasificación) de todos los árboles.
- Regla de oro:** El Random Forest reduce la **varianza** del modelo sin aumentar significativamente el sesgo.
-

4. El Pipeline de Preprocesamiento

En el código, utilizamos un `ColumnTransformer`. Esto no es solo por orden, sino por **integridad científica**:

- **OneHotEncoder:** Los árboles no entienden "North" o "South" de forma nativa; necesitan variables *dummy*.
 - **Evitar el Data Leakage:** Al meter el preprocesamiento dentro de un Pipeline, nos aseguramos de que las transformaciones se calculen correctamente durante la Validación Cruzada, evitando que información del set de validación se "filtre" al entrenamiento.
-

5. Evaluación Robusta

¿Por qué F1-Score y no Accuracy?

En problemas de Churn, los datos suelen estar desbalanceados (hay más gente que se queda que la que se va).

- **Accuracy:** Puede engañarte. Si el 90% se queda, un modelo que diga "nadie se va" tiene 90% de precisión pero es inútil.
- **F1-Score:** Es la media armónica entre *Precision* y *Recall*. Nos obliga a ser buenos detectando a los que se van sin disparar demasiadas falsas alarmas.

Validación Cruzada (Cross-Validation)

No confiamos en una sola partición de datos. El `StratifiedKFold` divide el dataset en K partes (folds) manteniendo la proporción de Churn en cada una, dándonos una métrica de rendimiento **estable y realista**.

6. Interpretabilidad y Feature Importance

Uno de los mayores valores de esta unidad es pasar de la predicción a la **acción**.

- **Feature Importance:** Nos dice qué variables reducen más la impureza en el bosque.
- **Acción de Negocio:** Si la "antigüedad" (`tenure_years`) es la variable más importante, el equipo de marketing debe crear campañas de fidelización para clientes nuevos.

3 - Pipelines en scikit-learn y buenas prácticas

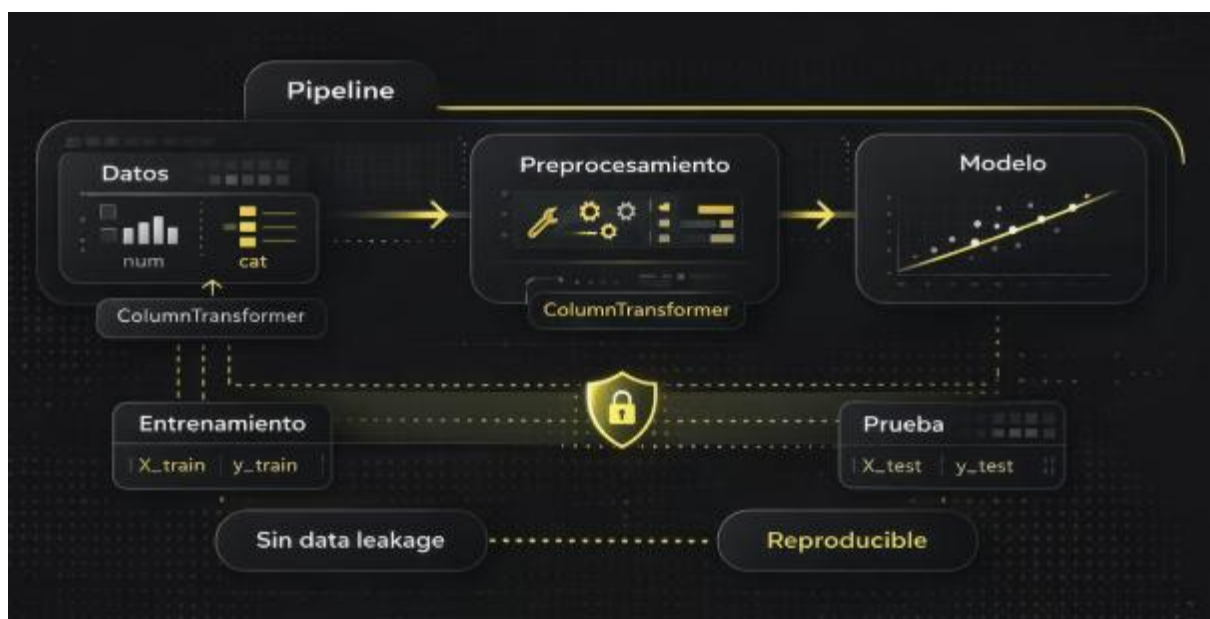
Este contenido introduce el concepto de pipelines en scikit-learn, su estructura y ventajas para crear flujos de trabajo reproducibles y evitar fugas de datos (data leakage). Se explican los componentes clave como Pipeline y ColumnTransformer, y se abordan problemas comunes de leakage y cómo prevenirlos, preparando al alumno para optimizar modelos y realizar búsquedas de hiperparámetros de forma segura y eficiente.

¿Por qué usar pipelines en machine learning?

Imagina que tienes un conjunto de datos con variables numéricas y categóricas, y quieres construir un modelo predictivo. ¿Cómo aseguras que el proceso de limpieza, transformación y modelado sea reproducible, ordenado y libre de errores? Aquí es donde los **pipelines** juegan un papel fundamental.

En este módulo, descubrirás cómo los pipelines en scikit-learn te permiten encadenar transformaciones y modelos en un flujo de trabajo único y coherente. Además, aprenderás a evitar uno de los problemas más comunes en machine learning: el **data leakage** o fuga de datos, que puede invalidar tus resultados.

Al finalizar esta unidad, serás capaz de estructurar pipelines con Pipeline y ColumnTransformer, y entenderás cómo estas herramientas facilitan la reproducibilidad y el mantenimiento de tus proyectos de análisis y ciencia de datos.



¿Qué es un Pipeline en scikit-learn?

Un `Pipeline` es un objeto que permite encadenar una secuencia de pasos, donde cada paso es una transformación o un estimador (modelo). Esto facilita:

- **Reproducibilidad:** el flujo completo se ejecuta de forma consistente.
- **Mantenimiento:** el código es más limpio y modular.
- **Prevención de errores:** evita aplicar transformaciones fuera de lugar.

Estructura básica de un Pipeline

Un pipeline se define como una lista ordenada de tuplas (nombre, transformador/estimador). Por ejemplo:

```
python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression())
])
```

ColumnTransformer: preprocesamiento selectivo

Cuando trabajamos con datos heterogéneos (numéricos y categóricos), necesitamos aplicar transformaciones diferentes a cada tipo. `ColumnTransformer` permite aplicar transformadores específicos a columnas seleccionadas:

```
python
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), ['edad', 'ingresos']),
    ('cat', OneHotEncoder(), ['sexo', 'ciudad'])
])
```

Luego, este preprocesador puede integrarse en un pipeline completo.

Ventajas clave

- Evita la **fuga de datos** al asegurar que las transformaciones se ajustan solo con los datos de entrenamiento dentro de cada fold de validación.

- Facilita la integración con métodos de búsqueda de hiperparámetros (GridSearchCV, RandomizedSearchCV).

¿Qué es el Data Leakage y por qué es un problema?

El **data leakage** ocurre cuando información del conjunto de prueba se filtra al conjunto de entrenamiento, generando evaluaciones optimistas y modelos que no generalizan.

Ejemplos comunes:

- Escalar o imputar datos antes de dividir en entrenamiento y prueba.
- Selección de características usando todo el dataset antes del entrenamiento.

Cómo evitarlo

- Usar pipelines para encadenar transformaciones y modelos.
- Ajustar transformadores solo con datos de entrenamiento dentro de cada iteración de validación.

***Nota:** scikit-learn maneja esto automáticamente cuando usas pipelines con validación cruzada o búsqueda de hiperparámetros.*

Resumen de conceptos clave

Concepto	Descripción breve
Pipeline	Secuencia ordenada de transformaciones y modelo
ColumnTransformer	Aplica transformaciones específicas por columna
Data Leakage	Fuga de información entre entrenamiento y prueba
Reproducibilidad	Capacidad de replicar resultados consistentemente

En la práctica profesional de analítica y ciencia de datos, los pipelines son esenciales para manejar proyectos complejos con múltiples etapas de preprocesamiento y modelado. Por ejemplo, en un proyecto de predicción de churn, podrías tener variables numéricas, categóricas y texto, cada una requiriendo transformaciones específicas.

Sin pipelines, el código puede volverse difícil de mantener y propenso a errores, especialmente cuando se realizan experimentos con diferentes modelos o parámetros.

Además, la prevención del data leakage es crítica para asegurar que los modelos que desarrollas sean confiables y generalicen bien a datos nuevos, evitando falsas expectativas que pueden afectar decisiones de negocio.

Este enfoque modular y reproducible también facilita la colaboración en equipos y la entrega de proyectos a producción, donde la consistencia es clave.

En las siguientes unidades, aplicarás estos conceptos para construir pipelines completos, optimizar hiperparámetros y preparar entregables reproducibles que reflejen las mejores prácticas de la industria.

4 - Principios de limpieza e imputación

Introducción a la detección y tratamiento de valores faltantes en datasets, explorando tipos de NA, estrategias simples y avanzadas de imputación, y las implicaciones estadísticas y de sesgo asociadas. Se enfatiza la toma de decisiones entre eliminar datos o imputarlos, con ejemplos prácticos en analítica de datos.

¿Alguna vez te has encontrado con un dataset incompleto y te has preguntado cómo manejar esos datos faltantes sin comprometer el análisis? En el mundo real de la analítica de datos, los valores faltantes son una constante y su manejo adecuado es crucial para construir modelos confiables y precisos. En esta unidad, exploraremos los principios fundamentales de la limpieza e imputación de datos, aprendiendo a identificar los diferentes tipos de valores faltantes y a aplicar estrategias efectivas para tratarlos. Además, discutiremos cuándo es preferible eliminar datos y cuándo es mejor imputarlos, siempre con un enfoque práctico y orientado a casos reales en ciencia de datos.



Limpieza e Imputación de Datos: Fundamentos

¿Qué son los valores faltantes?

Los valores faltantes (NA o NaN) son datos ausentes en un conjunto de datos que pueden afectar la calidad y resultados de cualquier análisis o modelo predictivo.

Tipos de valores faltantes

Es fundamental distinguir entre los tipos de valores faltantes para elegir la estrategia adecuada:

Tipo	Descripción	Ejemplo
MCAR (Missing Completely At Random)	La ausencia es completamente aleatoria y no depende de ninguna variable observable o no observable.	Datos perdidos por error de registro aleatorio.
MAR (Missing At Random)	La ausencia depende de variables observables, pero no del valor faltante en sí.	Edad faltante más común en ciertos grupos demográficos.

Tipo	Descripción	Ejemplo
MNAR (Missing Not At Random)	La ausencia depende del valor faltante mismo.	Personas con ingresos altos que no reportan su salario.

***Nota:** Identificar el tipo de valor faltante es clave para evitar sesgos en el análisis.*

Estrategias simples de limpieza e imputación

- **Eliminación de filas o columnas:**
- Útil cuando la cantidad de datos faltantes es pequeña.
- Riesgo: pérdida de información valiosa.
- **Imputación con estadísticos simples:**
- Media, mediana o moda según el tipo de variable.
- Fácil de implementar pero puede introducir sesgo.
- **Imputación con constantes o valores específicos:**
- Por ejemplo, imputar con cero o "desconocido".

Estrategias avanzadas y consideraciones estadísticas

- **Imputación basada en modelos:**
- Regresión, KNN, o técnicas más sofisticadas como MICE (Multiple Imputation by Chained Equations).
- **Imputación múltiple:**
- Genera varias imputaciones para reflejar la incertidumbre.
- **Escalabilidad y complejidad:**
- Considerar el costo computacional y la interpretabilidad.

Implicaciones estadísticas y de sesgo

- La imputación incorrecta puede introducir sesgos y afectar la validez del modelo.
- Es importante evaluar el impacto de la imputación mediante análisis exploratorios y validación.

Decidir entre eliminar o imputar

- Evaluar la proporción y patrón de valores faltantes.
- Considerar el impacto en la muestra y representatividad.
- Balancear entre pérdida de datos y riesgo de sesgo.

Con este conocimiento, estarás listo para aplicar técnicas de limpieza e imputación que mejoren la calidad de tus datos y la efectividad de tus modelos.

En la práctica de la ciencia de datos, los datasets reales suelen contener valores faltantes debido a errores en la recolección, problemas técnicos o simplemente porque ciertos datos no están disponibles. Por ejemplo, en un análisis de clientes, puede faltar información sobre ingresos o preferencias. Manejar estos valores faltantes adecuadamente es esencial para evitar que los modelos aprendan patrones erróneos o que los resultados sean poco confiables.

Este conocimiento es fundamental para preparar datasets que luego serán usados en modelos supervisados, como regresión o clasificación, que veremos en módulos posteriores. Además, entender cuándo eliminar datos o imputarlos impacta directamente en la calidad y reproducibilidad de tus análisis.

5 - Laboratorio: imputación y limpieza con pandas y scikit-learn

Ejercicio práctico para aplicar técnicas de limpieza e imputación de datos usando Python, pandas y scikit-learn. El alumno implementará funciones para detectar y tratar valores faltantes y outliers, y aplicará imputadores en pipelines, asegurando que las transformaciones sean robustas y no filtren datos de validación.

1. Contexto del problema

En problemas reales de Data Science, **los datos rara vez llegan completos y limpios**.

Valores faltantes (*missing values*) y valores extremos (*outliers*) son habituales debido a:

- errores de medición,
- fallas en sistemas,
- formularios incompletos,

- reglas de negocio (clientes nuevos, datos no aplicables),
- eventos atípicos reales.

El objetivo de este laboratorio es **reproducir un escenario realista de negocio**, aplicar **técnicas modernas de imputación**, y **evaluar cuantitativamente su calidad**, evitando errores metodológicos comunes como el *data leakage*.

2. Simulación de un dataset realista

Se parte de un dataset sintético que representa clientes de una empresa:

- **Variables numéricas:** edad, ingresos, antigüedad, cantidad de productos, ticket promedio, score crediticio.
- **Variables categóricas:** región, canal, segmento.
- **Variable objetivo:** gasto estimado el próximo mes.

¿Por qué simular datos?

- Permite conocer el **valor verdadero** antes de introducir missingness.
- Habilita una **evaluación objetiva** de la imputación (comparando imputado vs real).
- Reproduce relaciones multivariadas similares a un problema real.

⚠ **Regla clave:** `df_full` representa la “verdad” y **no debe modificarse**.

3. Mecanismos de valores faltantes (Missingness)

No todos los valores faltantes se comportan igual. En el laboratorio se introducen tres fenómenos:

3.1 MCAR — Missing Completely At Random

Los valores faltantes ocurren **sin depender de ninguna variable**.

Ejemplo:

un campo no se guardó por un error técnico aleatorio.

- ✓ Es el escenario más simple
 - ✓ Permite imputaciones básicas sin gran sesgo
 - ✗ Poco frecuente en la vida real
-

3.2 MAR — Missing At Random

La ausencia **depende de otra variable observable**.

Ejemplo:

clientes con alta antigüedad tienen mayor probabilidad de no declarar ingresos.

- ✓ Muy común en problemas reales
 - ✓ Métodos multivariados (KNN, MICE) suelen funcionar mejor
 - ✗ Imputaciones simples pueden introducir sesgo
-

3.3 MNAR — Missing Not At Random (no simulado explícitamente)

La ausencia depende del **valor faltante en sí** (ej: clientes con ingresos muy bajos no responden).

- ✗ Muy difícil de corregir solo con datos
 - ✗ Requiere conocimiento de dominio
-

4. Introducción y tratamiento de outliers

Los outliers pueden:

- distorsionar estadísticas como la media,
- afectar imputadores basados en distancia,
- generar imputaciones irreales.

Estrategia aplicada: IQR + Winsorization

1. Se detectan límites usando el método del **rango intercuartílico (IQR)**.
2. Se “recortan” valores extremos (*capping*) en lugar de eliminarlos.
3. El cálculo se hace **solo con datos de entrenamiento**.

☞ Esto preserva información sin introducir *data leakage*.

5. Separación Train / Test y Data Leakage

Antes de imputar:

- Se divide el dataset en **train y test**.
- Los imputadores se **ajustan solo con train**.
- Test se transforma usando esos parámetros.

¿Por qué?

Si el imputador “ve” el test durante el ajuste:

- aprende información del futuro,
- subestima el error,
- genera resultados artificialmente optimistas.

✦ Este es uno de los errores más comunes en proyectos reales.

6. Estrategias de imputación

6.1 SimpleImputer

- Numéricas: media
- Categóricas: moda

Ventajas

- Simple, rápido, robusto
- Buen baseline

Limitaciones

- No respeta relaciones entre variables
- Distorsiona la varianza
- Puede sesgar modelos predictivos

6.2 KNNImputer

Imputa cada valor faltante usando los *k vecinos más cercanos*.

Ventajas

- Aprovecha correlaciones locales
- Mejor que la media cuando hay estructura

Limitaciones

- Costoso computacionalmente
 - Sensible a escalado y outliers
 - No escala bien a datasets grandes
-

6.3 IterativeImputer (MICE)

Modela cada variable como función de las demás y repite el proceso iterativamente.

Ventajas

- Captura relaciones multivariadas
- Genera imputaciones más coherentes
- Suele ser el método más preciso

Riesgos

- Mayor complejidad
 - Puede sobreajustar
 - Requiere buen control metodológico
-

7. Evaluación de la imputación

Gracias a que conocemos los valores originales (`df_full`), podemos evaluar:

7.1 Variables numéricas

- **RMSE** solo en posiciones que originalmente eran NaN.

Esto responde a:

“¿Qué tan lejos quedó el valor imputado del verdadero?”

7.2 Variables categóricas

- **Accuracy** sobre posiciones faltantes.

Esto responde a:

“¿Qué proporción de categorías fue imputada correctamente?”

△ Nunca se evalúa sobre valores que no eran faltantes.

8. Comparación y análisis

Los resultados deben analizarse considerando:

- desempeño en **train vs test**,
- estabilidad entre métodos,
- costo computacional,
- facilidad de implementación.

No siempre el método más complejo es el más adecuado.

9. Visualización como herramienta de diagnóstico

Las distribuciones permiten detectar:

- sobre-concentración en la media (SimpleImputer),
- artefactos de suavizado (KNN),
- mayor coherencia estructural (MICE).

Las gráficas complementan la métrica numérica.

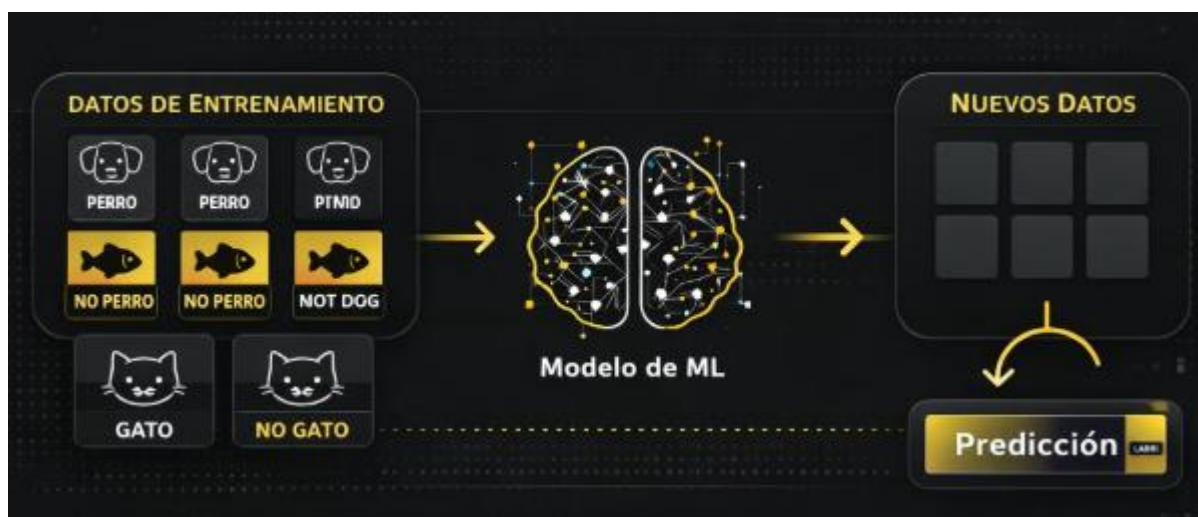
10. Buenas prácticas finales

- ✓ Separar train/test antes de imputar
- ✓ Identificar el mecanismo de missingness
- ✓ Tratar outliers antes de imputar
- ✓ Comparar siempre contra un baseline
- ✓ Documentar decisiones
- ✓ Evitar complejidad innecesaria

6 - ¿Qué es el aprendizaje supervisado?

En esta unidad, exploraremos el paradigma del aprendizaje supervisado, un pilar fundamental en la ciencia de datos y el machine learning. Aprenderás a diferenciar entre los dos tipos principales de problemas que aborda: clasificación y regresión, y conocerás ejemplos prácticos aplicados a contextos reales de negocio. Esta base te permitirá comprender cuándo y cómo aplicar cada enfoque en tus proyectos de análisis de datos.

¿Alguna vez te has preguntado cómo las aplicaciones de tu teléfono pueden reconocer si una foto es de un gato o un perro, o cómo los bancos deciden si aprobar un préstamo? Detrás de estas decisiones está el aprendizaje supervisado, un enfoque de machine learning que utiliza datos etiquetados para enseñar a los modelos a predecir resultados. En esta unidad, descubrirás qué es el aprendizaje supervisado, cómo se diferencia en sus dos grandes categorías — clasificación y regresión — y verás ejemplos concretos que te ayudarán a entender cuándo usar cada uno.



Aprendizaje Supervisado: Definición y Fundamentos

1. ¿Qué es el aprendizaje supervisado?

El **aprendizaje supervisado** es un enfoque de machine learning en el cual un modelo aprende a partir de un conjunto de datos que **incluye la respuesta correcta**, llamada *variable objetivo* o *label*. Cada observación del dataset está compuesta por:

- Un conjunto de **variables de entrada** XXX
- Una **salida conocida** YYY

El objetivo es que el modelo aprenda una función:

$f: X \rightarrow Y$ $f: X \rightarrow Y$

capaz de predecir correctamente la salida para nuevos datos que no ha visto durante el entrenamiento.

2. Clasificación vs Regresión

Dentro del aprendizaje supervisado existen dos grandes tipos de problemas, ambos abordados en esta práctica.

Clasificación

- **Qué se predice:** una categoría discreta.
- **Ejemplo del ejercicio:** churn (abandona / no abandona).
- **Tipo de variable objetivo:** binaria (0 o 1).
- **Modelos y métricas típicas:** Logistic Regression, Accuracy, Precision, Recall, ROC-AUC.

Regresión

- **Qué se predice:** un valor numérico continuo.
- **Ejemplo del ejercicio:** next_month_spend (gasto del próximo mes).
- **Tipo de variable objetivo:** numérica continua.
- **Modelos y métricas típicas:** Random Forest Regressor, MAE, RMSE, R².

Comprender esta diferencia es clave, ya que **determina el modelo, las métricas y la interpretación de resultados**.

3. Contexto del problema de negocio simulado

La práctica se basa en un **escenario realista de análisis de clientes**, donde se dispone de:

- Variables demográficas y financieras (edad, ingresos, score crediticio).
- Variables de comportamiento (antigüedad, cantidad de productos, ticket promedio).
- Variables categóricas (región, canal preferido, categoría favorita).

A partir de estas variables se plantean dos objetivos de negocio:

1. **Predecir churn** → problema de clasificación.
2. **Predecir gasto futuro** → problema de regresión.

Este enfoque permite demostrar cómo un mismo dataset puede ser utilizado para **distintos tipos de aprendizaje supervisado**, cambiando únicamente la variable objetivo.

4. Preparación de los datos (Preprocesamiento)

Antes de entrenar cualquier modelo supervisado, es fundamental preparar los datos correctamente.

4.1 Separación por tipo de variable

Se distinguen explícitamente:

- **Variables numéricas**
- **Variables categóricas**

Esto es necesario porque cada tipo requiere técnicas de transformación distintas.

5. Pipeline de preprocesamiento

El preprocesamiento se construye mediante pipelines independientes para cada tipo de variable, integrados con `ColumnTransformer`.

5.1 Tratamiento de variables numéricas

El pipeline numérico incluye:

a) Imputación con la mediana

- Evita errores por valores faltantes.
- Es robusta frente a outliers.

b) Escalado con `StandardScaler`

- Centra las variables en media 0 y desviación estándar 1.
 - Es esencial para modelos sensibles a la escala, como la regresión logística.
 - Garantiza que ninguna variable domine por su magnitud.
-

5.2 Tratamiento de variables categóricas

El pipeline categórico incluye:

a) Imputación con el valor más frecuente

- Mantiene la categoría dominante.
- Evita introducir categorías artificiales.

b) One-Hot Encoding

- Convierte categorías en variables binarias.
 - Evita imponer un orden artificial.
 - Permite que los modelos trabajen con datos categóricos.
 - `handle_unknown='ignore'` asegura robustez ante categorías nuevas.
-

6. Integración con ColumnTransformer

`ColumnTransformer` permite:

- Aplicar transformaciones distintas a cada grupo de columnas.
- Evitar errores manuales en la preparación de datos.
- Garantizar que el preprocesamiento sea **reproducible y consistente**.

Este paso es clave para trabajar de forma profesional en proyectos reales.

7. Construcción del pipeline de aprendizaje supervisado

El pipeline completo integra:

1. **Preprocesamiento**
2. **Modelo de machine learning**

Esto asegura:

- Ausencia de *data leakage*.
 - Reproducibilidad del experimento.
 - Facilidad para entrenar, evaluar y desplegar modelos.
-

8. Modelos utilizados y justificación

8.1 Clasificación — Logistic Regression

Se elige **Logistic Regression** porque:

- Es un excelente modelo baseline.
- Es rápido de entrenar.
- Sus coeficientes son interpretables.
- Permite entender cómo cada variable impacta en la probabilidad de churn.

Se evalúa con:

- Accuracy
- Precision
- Recall
- ROC-AUC
- Matriz de clasificación (classification report)
- Curva ROC

8.2 Regresión — Random Forest Regressor

Se utiliza **Random Forest Regressor** porque:

- Captura relaciones no lineales.
- Es robusto a outliers.
- Funciona bien sin supuestos fuertes sobre los datos.
- Permite obtener importancias de variables.

Se evalúa con:

- MAE (error absoluto medio)

- RMSE (error cuadrático medio)
- R^2 (proporción de varianza explicada)

9. Interpretabilidad de resultados

La práctica incluye interpretación de modelos:

- **Clasificación:** análisis de coeficientes de la regresión logística para identificar variables más influyentes.
- **Regresión:** análisis de importancias de variables del Random Forest.

Esto conecta el modelado con la **toma de decisiones de negocio**, uno de los objetivos centrales del aprendizaje supervisado.

10. Conclusión

Esta práctica demuestra de forma integral cómo:

- Identificar un problema de aprendizaje supervisado.
- Distinguir entre clasificación y regresión.
- Preparar datos numéricos y categóricos correctamente.
- Construir pipelines profesionales con scikit-learn.
- Entrenar, evaluar e interpretar modelos supervisados.

El enfoque presentado refleja **buenas prácticas reales en Data Science**, y constituye una base sólida para avanzar hacia modelos más complejos y escenarios productivos.

Clasificación vs Regresión

Dentro del aprendizaje supervisado, existen dos tipos principales de problemas:

Clasificación

- **¿Qué es?** Predecir una categoría o clase a la que pertenece un dato.
- **Ejemplos de etiquetas:** "spam" o "no spam", "aprobado" o "rechazado", tipos de enfermedades.
- **Características:** Las salidas son variables categóricas discretas.

Regresión

- **¿Qué es?** Predecir un valor numérico continuo.

- **Ejemplos de etiquetas:** precio de una casa, temperatura, ingresos mensuales.
- **Características:** Las salidas son variables numéricas continuas.

Aspecto	Clasificación	Regresión
Tipo de salida	Categorías discretas	Valores numéricos continuos
Ejemplo	Diagnóstico médico (enfermo/sano)	Predicción de precio de vivienda
Métricas comunes	Precisión, matriz de confusión	MAE, RMSE

***Nota:** Elegir correctamente entre clasificación y regresión es clave para aplicar el modelo adecuado y obtener resultados útiles.*

Ejemplos en la Industria

- **Clasificación:**
 - Detección de fraude en transacciones bancarias (fraude/no fraude).
 - Clasificación de correos electrónicos (spam/no spam).
 - Diagnóstico médico basado en síntomas (enfermedad A, B o C).
- **Regresión:**
 - Predicción del precio de viviendas según características.
 - Estimación de demanda energética diaria.
 - Pronóstico de ventas mensuales para un producto.

Estos ejemplos muestran cómo el aprendizaje supervisado se aplica para resolver problemas concretos y aportar valor en diferentes sectores.

En la práctica, entender si un problema es de clasificación o regresión te guía en la selección de algoritmos, preparación de datos y evaluación de modelos. Por ejemplo, para un problema de clasificación, usarás métricas como la matriz de confusión o el AUC-ROC, mientras que para regresión, métricas como el error cuadrático medio (RMSE) son más adecuadas.

Además, la naturaleza de la variable objetivo influye en cómo prepararás tus datos y qué técnicas de modelado aplicarás. Esta distinción es fundamental para diseñar soluciones efectivas en proyectos de analítica y ciencia de datos.

En las siguientes unidades, profundizaremos en cómo implementar modelos supervisados en Python, explorando algoritmos específicos y buenas prácticas para preparar tus datos y evaluar tus modelos.

7 - Visión general de modelos supervisados y cuándo usarlos

Este contenido ofrece un panorama claro y práctico de los modelos supervisados clásicos, destacando sus ventajas, limitaciones, complejidad e interpretabilidad, para que puedas seleccionar el modelo más adecuado según la naturaleza de tus datos y el problema a resolver.

¿Qué modelo supervisado elegir para tu problema de datos?

Imagina que tienes un conjunto de datos y necesitas predecir un resultado: ¿cómo decides qué modelo supervisado usar? En este módulo, exploraremos los modelos supervisados clásicos más comunes, sus características, ventajas y limitaciones, para que puedas tomar decisiones informadas y prácticas en tus proyectos de análisis y ciencia de datos.

Este recorrido te permitirá entender cuándo es mejor usar una regresión lineal frente a un árbol de decisión, o cuándo un modelo como Random Forest puede ser la mejor opción. Además, prepararemos el terreno para que puedas implementar y comparar estos modelos en los siguientes módulos, siempre con un enfoque práctico y orientado a resultados reales.



Modelos supervisados clásicos: una visión general

El aprendizaje supervisado consiste en construir modelos que, a partir de datos etiquetados, puedan predecir resultados para nuevos datos. Los modelos clásicos que veremos son:

- **Regresión Lineal:** Ideal para problemas de regresión donde la variable objetivo es continua. Es simple, rápido y fácil de interpretar.
- **Regresión Logística:** Usado para clasificación binaria, predice probabilidades de pertenencia a clases. También es interpretable y eficiente.
- **K-Nearest Neighbors (K-NN):** Clasifica o predice basándose en la cercanía a ejemplos conocidos. No requiere entrenamiento explícito, pero puede ser costoso en predicción.
- **Árboles de Decisión:** Modelos que segmentan el espacio de datos en regiones según reglas simples. Son interpretables y manejan bien datos mixtos.
- **Random Forest:** Ensamble de árboles que mejora la precisión y reduce el sobreajuste, a costa de menor interpretabilidad.

Complejidad y escalabilidad

Modelo	Complejidad de Entrenamiento	Complejidad de Predicción	Escalabilidad
Regresión Lineal	Baja	Muy baja	Alta
Regresión Logística	Baja	Baja	Alta
K-NN	Nula (pues no entrena)	Alta (depende de datos)	Baja
Árboles de Decisión	Moderada	Baja	Moderada
Random Forest	Alta (varios árboles)	Moderada	Moderada

Interpretabilidad

- **Alta:** Regresión Lineal, Regresión Logística, Árboles de Decisión
- **Media:** Random Forest

- **Baja:** K-NN (difícil explicar decisiones individuales)

La interpretabilidad es clave en sectores regulados o cuando se requiere explicar decisiones a stakeholders.

Ventajas y limitaciones resumidas

Modelo	Ventajas	Limitaciones
Regresión Lineal	Simple, rápido, interpretable	Solo relaciones lineales, sensible a outliers
Regresión Logística	Probabilidades, interpretable	Solo clasificación binaria, asume independencia
K-NN	Intuitivo, no paramétrico	Costoso en predicción, sensible a ruido
Árboles de Decisión	Fácil de interpretar, maneja variables mixtas	Puede sobreajustar, inestable
Random Forest	Preciso, reduce sobreajuste	Menos interpretable, más costoso

Aplicación práctica y selección de modelos

Al enfrentar un problema real, considera:

- **Tipo de variable objetivo:** ¿Es continua (regresión) o categórica (clasificación)?
- **Tamaño y dimensionalidad de los datos:** Modelos simples para datos pequeños, modelos más robustos para grandes volúmenes.
- **Necesidad de interpretabilidad:** ¿Debes explicar el modelo a terceros?
- **Recursos computacionales disponibles:** Algunos modelos requieren más tiempo y memoria.

Por ejemplo, para predecir ventas futuras (variable continua), la regresión lineal puede ser un buen punto de partida. Para clasificar correos como spam o no spam, la regresión logística o Random Forest pueden ser opciones adecuadas.

8 - Métricas en profundidad: clasificación y regresión

Explora en detalle las métricas clave para evaluar modelos de clasificación y regresión, incluyendo accuracy, precision, recall, F1, ROC/AUC, MAE y RMSE. Aprende a interpretar la matriz de confusión y las curvas ROC/AUC, y descubre estrategias para manejar conjuntos de datos con clases desbalanceadas, seleccionando métricas adecuadas según el objetivo de negocio.

¿Alguna vez te has preguntado cómo saber si un modelo de machine learning realmente funciona bien para tu problema? No basta con que prediga, sino que debemos medir su desempeño con métricas adecuadas que reflejen el objetivo de negocio y la naturaleza de los datos. En esta unidad, profundizaremos en las métricas más utilizadas para evaluar modelos de clasificación y regresión, y aprenderemos a interpretar herramientas clave como la matriz de confusión y las curvas ROC/AUC. Además, abordaremos un desafío común en la industria: los datos con clases desbalanceadas, y cómo elegir métricas y estrategias que permitan evaluaciones justas y efectivas. Al finalizar, estarás capacitado para seleccionar y aplicar métricas que guíen decisiones informadas en proyectos reales de analítica y ciencia de datos.

Métricas de evaluación para modelos supervisados

La evaluación de modelos es fundamental para entender su desempeño y utilidad. Dependiendo del tipo de problema — clasificación o regresión — se utilizan diferentes métricas.

Métricas para clasificación (REINFORCE)

- **Accuracy (Exactitud):** Proporción de predicciones correctas sobre el total. Útil cuando las clases están balanceadas.
- **Matriz de confusión:** Tabla que muestra los verdaderos positivos (TP), falsos positivos (FP), verdaderos negativos (TN) y falsos negativos (FN). Es la base para otras métricas.
- **Precision (Precisión):** Proporción de predicciones positivas correctas sobre todas las predicciones positivas.
- **Recall (Sensibilidad o Exhaustividad):** Proporción de casos positivos correctamente identificados sobre todos los casos positivos reales.
- **F1-Score:** Media armónica entre precision y recall, balancea ambos aspectos.
- **Curva ROC (Receiver Operating Characteristic):** Gráfica que muestra la tasa de verdaderos positivos vs tasa de falsos positivos para distintos umbrales.

- **AUC (Area Under the Curve):** Área bajo la curva ROC, mide la capacidad global del modelo para distinguir clases.

***Ejemplo:** En detección de fraude, un modelo con alta precisión evita falsos positivos (alertas innecesarias), mientras que un alto recall detecta la mayoría de fraudes reales.*

Métricas para regresión (REINFORCE)

- **MAE (Mean Absolute Error):** Promedio de las diferencias absolutas entre predicciones y valores reales. Fácil de interpretar en unidades originales.
- **RMSE (Root Mean Squared Error):** Raíz cuadrada del promedio de los errores al cuadrado. Penaliza más errores grandes.

***Ejemplo:** En predicción de precios, MAE indica el error promedio esperado, mientras RMSE destaca errores grandes que pueden ser críticos.*

Manejo de datos desbalanceados (INTRODUCE)

Cuando una clase es mucho más frecuente que otra, la accuracy puede ser engañosa (p. ej., predecir siempre la clase mayoritaria). Por eso:

- Se usan métricas como precision, recall y F1 que consideran el balance entre clases.
- Se aplican técnicas de re-muestreo: oversampling (aumentar minoría), undersampling (reducir mayoría).
- Se ajustan umbrales de decisión para optimizar trade-offs.

***Nota:** Elegir la métrica correcta depende del contexto y el impacto de errores tipo I y II en el negocio.*

Conoce más: Scikit-learn: Model Evaluation

En la práctica profesional, seleccionar la métrica adecuada es clave para que los modelos aporten valor real. Por ejemplo, en un sistema de detección de spam, un alto recall evita que correos legítimos sean marcados erróneamente, mientras que en un modelo de predicción de demanda, minimizar el error absoluto (MAE) puede traducirse en ahorros significativos.

La matriz de confusión y las curvas ROC/AUC son herramientas visuales que permiten entender el comportamiento del modelo más allá de un solo número, facilitando la comunicación con stakeholders y la toma de decisiones.

Además, los datos desbalanceados son frecuentes en sectores como salud, finanzas o seguridad, donde la clase de interés es rara pero crítica. Saber manejar estos casos con métricas y técnicas adecuadas es una habilidad esencial para un analista o científico de datos.

Esta unidad prepara el terreno para implementar pipelines de evaluación reproducible, que veremos en unidades posteriores, asegurando que las métricas elegidas se calculen y reporten de forma consistente y automatizada.

Para practicar

Instrucciones:

Les mostramos código de ejemplo para que vean como se utilizan las métricas:

```
# =====
# IMPORTACIÓN DE LIBRERÍAS
# =====
import numpy as np
import pandas as pd                    # Manejo de DataFrame
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression # Modelo simple
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score
from sklearn.metrics import recall_score, f1_score, roc_curve, auc
import matplotlib.pyplot as plt        # Para graficar la curva ROC
# =====
# CREACIÓN DEL DATASET
# =====
np.random.seed(42) # reproducibilidad
n = 200 # tamaño del dataset
# Horas de estudio: mayoría entre 0 y 12, pero agregamos errores
horas_estudio = np.random.normal(5, 2.5, n).clip(0, 15)
# Tutorías: 0 o 1 con cierto balance
tutorias = np.random.choice([0, 1], size=n, p=[0.4, 0.6])
# Probabilidad base de aprobar: depende de horas + tutorías
prob_aprobar = (
    0.05 * horas_estudio +
    0.25 * tutorias +
    np.random.normal(0, 0.15, n)
)
# Escalamos la probabilidad entre 0 y 1
prob_aprobar = (prob_aprobar - prob_aprobar.min()) / (prob_aprobar.max() - prob_aprobar.min())
# Variable objetivo base
aprobo = np.where(prob_aprobar > 0.5, 1, 0)
# -----
# AGREGAR ERRORES INTENCIONALES
# -----
```



```

# 1) Personas con muchas horas (>=12) pero NO aprueban
idx_muchas_horas = np.where(horas_estudio >= 12)[0]
if len(idx_muchas_horas) >= 3:
    for i in np.random.choice(idx_muchas_horas, size=3, replace=False):
        aprobo[i] = 0
# 2) Personas con pocas horas (<2) pero aprueban
idx_pocas_horas = np.where(horas_estudio < 2)[0]
if len(idx_pocas_horas) >= 3:
    for i in np.random.choice(idx_pocas_horas, size=3, replace=False):
        aprobo[i] = 1
# DataFrame final
df = pd.DataFrame({
    "horas_estudio": horas_estudio.round(2),
    "tutorias": tutorias,
    "aprobo": aprobo
})
# =====
# MOSTRAR DATASET
# =====
print("DATASET CREADO:")
print(df.head(20), "\n")
# =====
# SEPARACIÓN ENTRE FEATURES Y TARGET
# =====
X = df[["horas_estudio", "tutorias"]]
y = df["aprobo"]
# División de datos
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
# =====
# MODELO LOGÍSTICO
# =====
modelo = LogisticRegression()
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)
y_pred_proba = modelo.predict_proba(X_test)[: , 1]
# =====
# MÉTRICAS BÁSICAS
# =====
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
# =====
# RESULTADOS
# =====
print("=== MÉTRICAS CLAVES PARA CLASIFICACIÓN ===\n")
print(f"Accuracy: {acc:.3f}")
print(f"Precision: {prec:.3f}")
print(f"Recall: {rec:.3f}")
print(f"F1-Score: {f1:.3f}\n")
print("Matriz de Confusión (TN, FP / FN, TP):")
print(cm, "\n")

```

```
# =====
# CURVA ROC Y AUC
# =====
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
auc_value = auc(fpr, tpr)
print(f"AUC: {auc_value:.3f}\n")
# Graficar curva ROC
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f"AUC = {auc_value:.3f}")
plt.plot([0,1],[0,1], '--')
plt.xlabel("False Positive Rate (FPR)")
plt.ylabel("True Positive Rate (TPR)")
plt.title("Curva ROC")
plt.legend()
plt.grid()
plt.show()
```

Y ahora las graficamos:

```
import matplotlib.pyplot as plt
import seaborn as sns
# =====
# GRAFICAR MATRIZ DE CONFUSIÓN
# =====
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Predijo NO", "Predijo Sí"],
            yticklabels=["Real NO", "Real Sí"])
plt.title("Matriz de Confusión")
plt.xlabel("Predicción")
plt.ylabel("Valor Real")
plt.show()
# =====
# GRAFICAR ACCURACY
# =====
plt.figure(figsize=(4,3))
plt.bar(["Accuracy"], [acc], color="green")
plt.ylim(0,1)
plt.title("Accuracy")
plt.ylabel("Valor")
plt.show()
# =====
# GRAFICAR PRECISION
# =====
plt.figure(figsize=(4,3))
plt.bar(["Precision"], [prec], color="orange")
plt.ylim(0,1)
plt.title("Precision")
plt.ylabel("Valor")
plt.show()
# =====
# GRAFICAR RECALL
# =====
plt.figure(figsize=(4,3))
plt.bar(["Recall"], [rec], color="red")
plt.ylim(0,1)
plt.title("Recall")
```

```
plt.ylabel("Valor")
plt.show()
# =====
# GRAFICAR F1-SCORE
# =====
plt.figure(figsize=(4,3))
plt.bar(["F1-Score"], [f1], color="purple")
plt.ylim(0,1)
plt.title("F1-Score")
plt.ylabel("Valor")
plt.show()
```

9 - Laboratorio: cross-validation y estimación robusta

Ejercicio práctico para implementar técnicas de validación cruzada (k-fold CV) y validación anidada (nested CV) usando JavaScript. El objetivo es que el alumno comprenda cómo estimar el desempeño generalizable de modelos y la variabilidad de las métricas obtenidas, aplicando funciones similares a `cross_val_score` y `cross_validate` en un contexto simplificado.

Marco Teórico — Laboratorio de Regresión Lineal

Objetivo del laboratorio

El objetivo de este laboratorio es **aplicar un modelo de aprendizaje supervisado de tipo regresión** para resolver un problema de negocio realista: **estimar el precio de una vivienda a partir de sus características**.

A lo largo de la práctica, se busca que el estudiante:

- Comprenda cuándo un problema es de **regresión**
- Aplique correctamente el flujo típico de un modelo supervisado
- Interprete métricas de error
- Analice los coeficientes del modelo para obtener conclusiones de negocio

1 Comprensión del problema: ¿por qué regresión?

En este caso, la variable objetivo es **el precio de una vivienda**, que es un **valor numérico continuo**.

Esto define claramente el problema como uno de **regresión**, ya que:

- No se busca clasificar en categorías

- Se desea predecir un valor real (por ejemplo, \$150.000)

☞ Por lo tanto, un modelo de **regresión lineal** es una primera aproximación razonable.

2 Generación del dataset sintético

En la práctica se utiliza un **dataset sintético**, lo cual es habitual en entornos educativos para:

- Controlar las relaciones entre variables
- Evitar problemas de calidad de datos reales
- Comprender mejor el comportamiento del modelo

Variables predictoras (X)

Las variables simuladas representan atributos típicos del mercado inmobiliario:

- `tamaño_m2`: superficie de la vivienda
- `habitaciones`: cantidad de habitaciones
- `antigüedad_años`: años desde la construcción

Estas variables se eligen porque **tienen una relación lógica con el precio**, lo cual facilita la interpretación posterior.

Variable objetivo (y)

- `precio`: se construye como una **combinación lineal** de las variables anteriores más un término de **ruido aleatorio**, para simular la variabilidad real del mercado.

Esto permite modelar un escenario realista donde:

- El tamaño y las habitaciones aumentan el precio
- La antigüedad lo reduce
- Existen factores no observados (ruido)

3 Preparación de variables (X e y)

Antes de entrenar cualquier modelo supervisado, es fundamental separar:

- **X (features)**: variables predictoras
- **y (target)**: variable que queremos predecir

Esta separación es clave porque:

- El modelo aprende una relación matemática entre X e y
- Permite evaluar el desempeño de forma correcta

☞ En esta práctica, los estudiantes deben identificar explícitamente qué columnas cumplen cada rol.

4 División Train / Test

Para evaluar si el modelo **generaliza correctamente**, se divide el dataset en:

- **Conjunto de entrenamiento (80%)**
- **Conjunto de test (20%)**

Esta división permite:

- Entrenar el modelo solo con una parte de los datos
- Evaluarlo sobre datos no vistos

Esto simula el escenario real donde el modelo debe predecir nuevos casos.

5 Entrenamiento del modelo de Regresión Lineal

La **regresión lineal** asume que existe una relación del tipo:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

Donde:

- Cada coeficiente representa el impacto de una variable
- El intercepto (β_0) es el valor base cuando todas las variables valen cero

Este modelo se elige porque:

- Es simple
- Es interpretable
- Funciona como baseline en problemas de regresión

6 Predicción y métricas de evaluación

Una vez entrenado el modelo, se generan predicciones sobre el conjunto de test y se evalúa el error usando métricas estándar de regresión:

Métricas utilizadas

- **MAE (Mean Absolute Error)**
Error promedio en unidades reales (pesos)
 - **MSE (Mean Squared Error)**
Penaliza más los errores grandes
 - **RMSE (Root Mean Squared Error)**
Error típico esperado (misma unidad que el target)
 - **R² (Coeficiente de determinación)**
Proporción de la variabilidad del precio explicada por el modelo
- Estas métricas permiten evaluar tanto:

- Precisión
- Estabilidad
- Capacidad explicativa

7 Interpretación de coeficientes

Una de las principales ventajas de la regresión lineal es su **interpretabilidad**.

Cada coeficiente indica:

- Cuánto cambia el precio esperado ante un aumento unitario de esa variable
- Manteniendo las demás constantes

Ejemplo conceptual:

- Si el coeficiente de `tamaño_m2` es 1200, significa que **cada metro cuadrado adicional aumenta el precio en promedio \$1200**

Ordenar los coeficientes por su valor absoluto permite identificar:

- Las variables con mayor impacto
- Qué factores son más relevantes desde el punto de vista del negocio

8 Gráficos de diagnóstico

Los gráficos ayudan a validar los supuestos del modelo:

Actual vs Predicho

- Permite visualizar qué tan cerca están las predicciones de los valores reales
- Idealmente los puntos deberían alinearse con la diagonal

Análisis de residuales

- Un modelo adecuado debería mostrar residuales:
- Centrados en cero
- Sin patrones evidentes

Si aparecen patrones, puede indicar:

- Relaciones no lineales
- Variables omitidas
- Necesidad de modelos más complejos

9 Reflexión final

- Justificar qué variable tiene mayor impacto
- Evaluar si la regresión lineal es suficiente
- Reconocer limitaciones del modelo

Esto refuerza que **el modelado no termina en el código**, sino en la interpretación y toma de decisiones.

Copía y pega el código en tu entorno de desarrollo para ver el flujo del proceso de modelo de regresión lineal.

10 - Visualización para diagnóstico y métricas (matplotlib, seaborn)

Ejercicio práctico para reforzar la creación de visualizaciones clave en la evaluación de modelos supervisados, utilizando matplotlib y seaborn. El alumno implementará funciones para graficar matrices de confusión, curvas ROC, scatter plots de residuos y distribuciones de errores, facilitando el diagnóstico visual y la interpretación de resultados.

1 Introducción

En machine learning, entrenar un modelo no es el final del proceso: es el comienzo del análisis crítico.

La visualización cumple un rol central en:

- Diagnosticar desempeño.
- Detectar errores sistemáticos.
- Identificar sesgos.
- Evaluar capacidad de discriminación.
- Comunicar resultados de forma clara.

Las métricas numéricas (accuracy, precision, recall, MSE, AUC, etc.) son necesarias, pero muchas veces no son suficientes. Las visualizaciones permiten comprender el comportamiento interno del modelo más allá de un único valor agregado.

En este módulo se utilizarán:

- **matplotlib** → biblioteca base para visualización en Python.
 - **seaborn** → capa de alto nivel construida sobre matplotlib, optimizada para análisis estadístico.
-

2 Rol de la visualización en modelos supervisados

Los modelos supervisados pueden dividirse en:

- Clasificación
- Regresión

Cada tipo requiere herramientas visuales específicas para evaluar su comportamiento.

Las visualizaciones permiten responder preguntas como:

- ¿Dónde se equivoca el modelo?
- ¿Confunde clases específicas?
- ¿Tiene buen poder discriminativo?
- ¿Existen patrones en los residuos?
- ¿Hay heterocedasticidad?
- ¿La distribución de errores es simétrica?

3 Matriz de Confusión (Clasificación)

◆ Concepto

La matriz de confusión resume el desempeño de un modelo de clasificación comparando:

- Valores reales
- Valores predichos

Estructura básica binaria:

	Predicción Positiva	Predicción Negativa
Real Positivo	True Positive (TP)	False Negative (FN)
Real Negativo	False Positive (FP)	True Negative (TN)

◆ ¿Por qué visualizarla?

Un valor de accuracy puede ocultar:

- Desbalance de clases.
- Errores críticos en una clase específica.
- Alta tasa de falsos negativos.

El heatmap permite:

- Ver rápidamente concentración de errores.
- Detectar patrones de confusión.
- Evaluar desempeño por clase.

Seaborn es especialmente útil para representar matrices de confusión como mapas de calor.

4 Curva ROC y AUC

◆ ROC (Receiver Operating Characteristic)

La curva ROC muestra la relación entre:

- Tasa de verdaderos positivos (TPR / Recall)
- Tasa de falsos positivos (FPR)

A distintos umbrales de decisión.

◆ Interpretación

- Curva cercana a la diagonal → modelo sin capacidad predictiva.

- Curva cercana al eje superior izquierdo → modelo fuerte.
 - Cuanto mayor el área bajo la curva (AUC), mejor el modelo discrimina entre clases.
-

◆ AUC (Area Under the Curve)

- Valor entre 0 y 1.
- 0.5 → modelo aleatorio.
- 1.0 → modelo perfecto.

La visualización permite:

- Comparar múltiples modelos.
 - Evaluar sensibilidad a umbrales.
 - Analizar trade-off entre recall y falsos positivos.
-

5 Residuos en Modelos de Regresión

◆ ¿Qué son los residuos?

Residuo = Valor real – Valor predicho

Representan el error individual por observación.

◆ Scatter Plot de Residuos

Un gráfico de residuos vs valores predichos permite detectar:

- Heterocedasticidad (varianza no constante).
- Patrones no lineales.
- Outliers.
- Problemas de especificación del modelo.

Idealmente:

- Los residuos deberían distribuirse aleatoriamente alrededor de cero.
- No deberían mostrar patrones sistemáticos.

Si se observan curvas o estructuras, puede indicar:

- Modelo mal especificado.
 - Falta de variables.
 - Relación no lineal no capturada.
-

6 Distribución de Errores

◆ Histograma de residuos

Permite analizar:

- Simetría.
- Sesgo.
- Presencia de colas largas.
- Outliers extremos.

En regresión lineal clásica se asume:

- Distribución aproximadamente normal de errores.

Si la distribución:

- Es asimétrica → puede haber sesgo sistemático.
- Tiene colas pesadas → posible presencia de outliers.
- Está desplazada de cero → modelo sistemáticamente sobreestima o subestima.

Seaborn facilita visualizar histogramas con estimaciones de densidad (KDE).

7 Diferencia entre matplotlib y seaborn

◆ matplotlib

- Control total y granularidad.
- Base para casi todas las visualizaciones en Python.
- Más flexible pero más verboso.

◆ seaborn

- Orientado a análisis estadístico.
- Sintaxis más simple.
- Mejor estética por defecto.
- Integración natural con pandas.

En entornos de diagnóstico de modelos:

- seaborn es ideal para heatmaps y distribuciones.
- matplotlib es útil para curvas ROC y gráficos personalizados.

8 Diagnóstico Visual vs Métricas Numéricas

Ejemplo:

Un modelo puede tener 90% accuracy, pero:

- Confundir sistemáticamente la clase minoritaria.
- Tener alta tasa de falsos negativos.
- Mostrar fuerte heterocedasticidad.

Las visualizaciones permiten detectar estos problemas que no siempre aparecen en una sola métrica agregada.

9 Buenas Prácticas en Visualización de Modelos

- ✓ Etiquetar correctamente ejes.
 - ✓ Incluir títulos descriptivos.
 - ✓ Indicar métricas relevantes (ej. AUC en la curva ROC).
 - ✓ Mantener consistencia visual.
 - ✓ No sobrecargar gráficos.
 - ✓ Separar visualizaciones por tipo de análisis.
-

10 Flujo de Diagnóstico Típico

En clasificación:

1. Matriz de confusión.
2. Precision / Recall.

3. Curva ROC.
4. AUC.
5. Análisis por clase.

En regresión:

1. MAE / MSE / RMSE.
 2. Scatter real vs predicho.
 3. Scatter de residuos.
 4. Histograma de residuos.
 5. Detección de outliers.
-

11 Importancia en Entornos Profesionales

En producción, la visualización es clave para:

- Validación de modelos antes de despliegue.
- Reportes ejecutivos.
- Auditoría de modelos.
- Monitoreo continuo.
- Detección de drift.

Un modelo sin diagnóstico visual es un modelo parcialmente evaluado.

12 Conexión con Ciencia de Datos Moderna

La visualización es fundamental en:

- Model risk management.
 - Explainability.
 - Model validation frameworks.
 - Regulación en IA.
 - Comunicación con stakeholders no técnicos.
-

🔗 Conclusión

Las visualizaciones con matplotlib y seaborn no son solo herramientas gráficas, sino instrumentos de diagnóstico profundo.

Permiten:

- Interpretar métricas.
- Detectar fallas estructurales.
- Comprender errores.
- Evaluar robustez.
- Comunicar resultados.

Dominar estas herramientas es esencial para cualquier profesional que trabaje con modelos supervisados en contextos reales.

Este ejercicio práctico te guiará en la creación de visualizaciones esenciales para evaluar y diagnosticar modelos supervisados. Usaremos matplotlib y seaborn para graficar matrices de confusión, curvas ROC, scatter plots de residuos y distribuciones de errores.

Objetivos

- Implementar funciones para graficar la matriz de confusión como heatmap
 - Crear la curva ROC y calcular el AUC
 - Visualizar residuos con scatter plot
 - Analizar la distribución de errores con histogramas
-

Instrucciones

Implementa las funciones indicadas en el código. Cada función recibirá datos simulados y debe devolver la visualización correspondiente. Al final, imprime los valores calculados para validación.

Nota: Este notebook está diseñado para reforzar el uso de matplotlib y seaborn en análisis de resultados de modelos supervisados, facilitando la interpretación visual y el diagnóstico efectivo.

11 - Escalado, normalización y codificación de variables

Ejercicio práctico para implementar escalado y codificación de variables usando técnicas comunes en preprocesamiento de datos, asegurando que las transformaciones se apliquen correctamente sin filtrar datos de validación.

1. Contexto del problema

En proyectos reales de Data Science, **la mayor parte del tiempo no se dedica a entrenar modelos**, sino a **entender, limpiar y preparar los datos**.

Los datasets de negocio rara vez están “limpios”: contienen valores faltantes, errores humanos, inconsistencias, duplicados y valores extremos que pueden distorsionar por completo un modelo.

Este laboratorio simula un **dataset de clientes** con un objetivo de negocio concreto:

☞ **predecir churn (abandono) el próximo mes.**

Antes de entrenar cualquier modelo, es imprescindible garantizar que los datos:

- representen correctamente la realidad,
 - no introduzcan sesgos artificiales,
 - y no contengan errores que afecten el aprendizaje del modelo.
-

2. Exploración inicial de datos (EDA mínima)

¿Por qué explorar antes de limpiar?

Nunca se debe limpiar un dataset “a ciegas”.

La exploración inicial permite:

- entender el tamaño del dataset,
- detectar columnas problemáticas,
- dimensionar la magnitud de los errores.

Qué se espera en esta etapa

- Ver el **shape** del dataset
- Visualizar las **primeras filas**
- Contar valores faltantes por columna

Esto permite responder preguntas como:

- ¿Qué variables tienen más missing?
- ¿Los problemas están concentrados o distribuidos?
- ¿Hay columnas que podrían descartarse?

★ **Principio clave:** *no se toman decisiones todavía, solo se observa.*

3. Valores ilegales y reglas de negocio

¿Qué es un valor ilegal?

Un valor ilegal es aquel que **no puede existir en el mundo real**, por ejemplo:

- edad negativa
- antigüedad de 10.000 meses

- ingresos negativos (según contexto)

Estos valores no son “outliers estadísticos”, sino **errores de carga o sistema**.

Por qué deben corregirse antes que todo lo demás

- Distorsionan estadísticas básicas (media, desvío).
- Rompen escaladores y modelos.
- Introducen ruido artificial.

Estrategias típicas

- Reemplazar por NaN para imputar luego.
- Corregir usando reglas de negocio (ej. clip).
- Eliminar registros solo si el daño es irreparable.

★ **Regla:** los valores ilegales se corrigen **antes** de imputar o escalar.

4. Normalización y estandarización de variables categóricas

El problema de las categóricas “sucias”

En datos reales, las categorías suelen venir con:

- espacios extra (" north ")
- diferencias de mayúsculas ("mobile" vs "Mobile")
- variantes de escritura ("Credit_Card" vs "credit_card")

Para un humano son iguales.

Para un modelo, **son categorías distintas**.

Riesgos de no normalizarlas

- Explosión artificial de cardinalidad.
- One-hot encoding incorrecto.
- Pérdida de información real.

Buenas prácticas

- Eliminar espacios (strip)
- Normalizar casing (lower o title)

- Unificar valores equivalentes explícitamente

✦ **Principio clave:** *una categoría = un significado único.*

5. Outliers: cuándo tratarlos y cuándo no

Outliers vs valores ilegales

- **Valores ilegales:** no deberían existir → se corrigen sí o sí.
- **Outliers:** existen, pero son extremos → requieren criterio.

Ejemplo:

- Un cliente con ingreso muy alto → posible.
- Un cliente con ingreso 12× mayor que el resto → sospechoso.

Por qué son peligrosos

- Afectan medias y varianzas.
- Distorsionan escaladores (StandardScaler).
- Influyen excesivamente en modelos lineales.

Estrategia usada en la práctica

- Detectar outliers con **IQR**.
- Aplicar **capping** / **winsorization**:
- se limita el valor, no se elimina el registro.

✦ **Ventaja:** se conserva información sin permitir que domine el modelo.

6. Duplicados: más que filas repetidas

Qué son duplicados parciales

En bases reales, un mismo cliente puede aparecer más de una vez:

- errores de carga
- múltiples fuentes
- problemas de sincronización

No siempre es correcto eliminar duplicados automáticamente.

Criterios posibles

- Mantener el registro más reciente.
- Mantener el de mayor calidad.
- Eliminar duplicados exactos.

✦ **Lo importante** no es la decisión, sino **justificarla**.

7. Missing values: entender antes de imputar

Tipos de missingness presentes

- **MCAR** (Missing Completely At Random)
→ valores faltantes sin patrón claro.
- **MAR** (Missing At Random)
→ el missing depende de otra variable
(ej. ingresos faltantes en clientes con mucha antigüedad).

Por qué esto importa

La estrategia de imputación depende del mecanismo:

- MCAR → media, mediana, moda pueden funcionar.
- MAR → imputaciones simples pueden introducir sesgo.

En este laboratorio

- Variables numéricas y categóricas se imputan **por separado**.
- Se prioriza claridad y buenas prácticas antes que complejidad.

✦ **Regla de oro:** imputar **después** de corregir outliers y valores ilegales.

8. Dataset final y validación

Antes de usar el dataset en ML, se debe verificar:

- No hay NaN.
- Las variables tienen rangos razonables.

- Las categorías están limpias y consistentes.
- El target no fue alterado indebidamente.

Este paso asegura que el dataset:

- es reproducible,
- es confiable,
- está listo para un pipeline.

9. Data leakage: el enemigo silencioso

Qué es data leakage

Ocurre cuando se usa información del futuro o del conjunto de validación para limpiar o entrenar el modelo.

Ejemplos en este laboratorio

- Calcular estadísticos usando todo el dataset antes de separar train/test.
- Imputar usando información del target.
- Normalizar usando valores que incluyen test.

✦ **Principio clave:** *todo lo que “aprende” debe hacerlo solo con train.*

10. Reflexión final

Este laboratorio no busca una única solución “correcta”, sino evaluar:

- criterio,
- razonamiento,
- justificación de decisiones.